Isaac Fredricks

CPSI 28003 - 01 and 9S1 Algorithms

Fall 2025

Please submit Homework 2 to the Blackboard

Department of Computer Science
University of Arkansas at Little Rock
2801 South University Avenue
Little Rock, Arkansas 72204-1099

Homework 2 (No Late Homework)
Due on **10/9/2022** at the beginning of the class
Mastering Asymptotic Notations in non-recursive programs

Question 1: Because the definition of big O is T(n) <= cg(n) for some constant c and integer n >= n0, for T(n)=2^n+1 and O(2^n), c would have to be 2, and n0 would have to be 1. For example, when n=2, T(2)=8 and 2g(2)=8, satisfying big-O. Another example would be n=3, where T(3)=16 and 2g(3)=16. This satisfies big-O notation

2 pages
5 Questions
Total Points: 100

1. (10 points) (Formally) Prove or disprove T(n) = 2^{n+1} is O(2^n) by using the definitions of the notations involved or disprove it by giving a counterexample.

2. (50 Points) Complexity analysis. What is the order of growth of the worst-case running time of the following code fragments? n is the input size.

(a) (10 Points)

```
for (int i=0; i<n; i++)
{
  for (int j = i; j<n; j++)
     S; // S is O(1);
}
```

O(n^2)

(b) (10 points)

```
int i = n;
while (i > 1)
{
  S; // Let assume S's O(1)
  i = i/2;
};
```

O(log base= 2 (n))

(c) (10 points)

```
for (int i=0; i<n; i++)
{
 for (int j = i; j<n; j++)
 S; // S is O(1);
}
```

O(n^2)

Insert text here

(d) (10 points)

```
int i = n;
while (i > 1)
{
  j = i;
  while (j < n)
  {
    S; // Let assume S's O(1)
    j = j * 2;
  };
  i = i / 2;
}
```

answer: O(n*log base=2 (n))

n * (log base 2 of n)

log base 2 of n

(e) (10 points)

```
int i = n;
while (i > 1000)
{
  S; // Let assume S's O(1)
  i = i*2;
};
```

O(nlog base=2 (n))

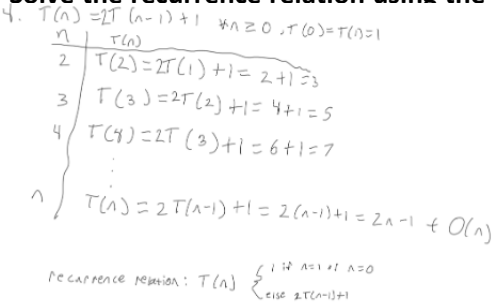
compiler error. Whenever n <= 1000, it iterates 0 times. When i > 1000, it iterates infinitely.

3. (10 Points) What is Big-O?
   (a) $n(log_{2^n})^2 + n(log_{2^n})$
   (b) $n^{1.1} + 2n^{1.75}$

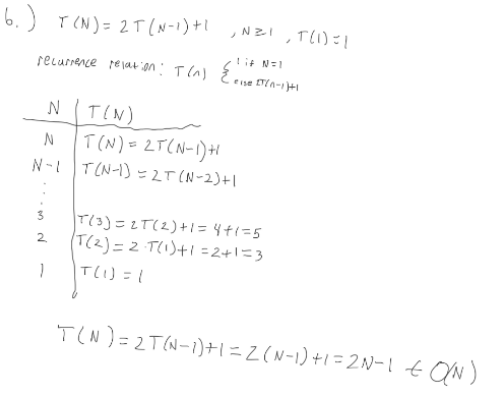
a. O(nlog base=2 (n))

b O(n^1.75)

4. (20 Points) **You need to show your work to get the credits. Solve the recurrence relation using the <u>forward/backward substitution</u>.**
   (a) (10 Points) **(Show the work.)**
   T(n) = T(n-1) + 1 for all n $\geq$ 0 and T(0) = T(1) = 1



   (b) (10 Points) **You need to show your work to get the credits. Solve the recurrence relation using <u>the forward/backward substitution</u>. (Show the work.)**
   T(N) = 2T(N-1) +1, N ≥ 1, T(1) = 1



5. (10 Points) Use the master method to solve the following recurre

   T(n) = 4T(2n/3) + n^2

   a. (2 Points) What is the value of a?
   b. (2 Points) What is the value of b?
   c. (2 Points) What is the value of d?
   d. (4 Points) What is the Big-O?

a. 4
b. 6
c. 2
d. O(n^2)